Creating Numbers/images with AI: A Hands-on Diffusion Model Exercise

Introduction

In this assignment, you'll learn how to create an AI model that can generate realistic images from scratch using a powerful technique called 'diffusion'. Think of it like teaching AI to draw by first learning how images get blurry and then learning to make them clear again.

What We'll Build

- A diffusion model capable of generating realistic images
- For most students: An AI that generates handwritten digits (0-9) using the MNIST dataset
- For students with more computational resources: Options to work with more complex datasets
- Visual demonstrations of how random noise gradually transforms into clear, recognizable images
- By the end, your AI should create images realistic enough for another AI to recognize them

Dataset Options

This lab offers flexibility based on your available computational resources:

- Standard Option (Free Colab): We'll primarily use the MNIST handwritten digit dataset, which works well with limited GPU memory and completes training in a reasonable time frame. Most examples and code in this notebook are optimized for MNIST.
- Advanced Option: If you have access to more powerful GPUs (either through Colab Pro/Pro+ or your own hardware), you can experiment with more complex datasets like Fashion-MNIST, CIFAR-10, or even face generation. You'll need to adapt the model architecture, hyperparameters, and evaluation metrics accordingly.

Resource Requirements

- Basic MNIST: Works with free Colab GPUs (2-4GB VRAM), ~30 minutes training
- Fashion-MNIST: Similar requirements to MNIST CIFAR-10: Requires more memory (8-12GB VRAM) and longer training (~2 hours)
- Higher resolution images: Requires substantial GPU resources and several hours of training

Before You Start

1. Make sure you're running this in Google Colab or another environment with GPU access
2. Go to 'Runtime' → 'Change runtime type' and select 'GPU' as your hardware accelerator
3. Each code cell has comments explaining what it does
4. Don't worry if you don't understand every detail - focus on the big picture!
5. If working with larger datasets, monitor your GPU memory usage carefully

The concepts you learn with MNIST will scale to more complex datasets, so even if you're using the basic option, you'll gain valuable knowledge about generative AI that applies to more advanced applications.

Step 1: Setting Up Our Tools

First, let's install and import all the tools we need. Run this cell and wait for it to complete.

```
# Step 1: Install required packages
%pip install einops
print("Package installation complete.")

# Step 2: Import libraries
# --- Core PyTorch libraries ---
import torch  # Main deep learning framework
import torch.nn.functional as F  # Neural network functions like activation functions
import torch.nn as nn  # Neural network building blocks (layers)
from torch.optim import Adam  # Optimization algorithm for training

# --- Data handling ---
from torch.utils.data import Dataset, DataLoader  # For organizing and loading our data
import torchvision  # Library for computer vision datasets and models
import torchvision.transforms as transforms  # For preprocessing images

# --- Tensor manipulation ---
import random  # For random operations
```

```
from einops.layers.torch import Rearrange  # For reshaping tensors in neural networks
from einops import rearrange  # For elegant tensor reshaping operations
import numpy as np  # For numerical operations on arrays

# --- System utilities ---
import os  # For operating system interactions (used for CPU count)

# --- Visualization tools ---
import matplotlib.pyplot as plt  # For plotting images and graphs
from PIL import Image  # For image processing
from torchvision.utils import save_image, make_grid  # For saving and displaying image grids

# Step 3: Set up device (GPU or CPU)
# Force CUDA errors to be synchronous for easier debugging
os.environ['CUDA_LAUNCH_BLOCKING'] = '1'
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"We'll be using: {device}")

# Check if we're actually using GPU (for students to verify)
if device.type == "cuda":
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
else:
    print("Note: Training will be much slower on CPU. Consider using Google Colab with GPU enabled.")
```

```
Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (0.8.2)
Package installation complete.
We'll be using: cuda
GPU name: Tesla T4
GPU memory: 15.64 GB
```

REPRODUCIBILITY AND DEVICE SETUP

```
# Step 4: Set random seeds for reproducibility
# Diffusion models are sensitive to initialization, so reproducible results help with debugging
SEED = 42  # Universal seed value for reproducibility
torch.manual_seed(SEED)          # PyTorch random number generator
np.random.seed(SEED)             # NumPy random number generator
random.seed(SEED)                # Python's built-in random number generator

print(f"Random seeds set to {SEED} for reproducible results")

# Configure CUDA for GPU operations if available
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)       # GPU random number generator
    torch.cuda.manual_seed_all(SEED)   # All GPUs random number generator

    # Ensure deterministic GPU operations
    # Note: This slightly reduces performance but ensures results are reproducible
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    try:
        # Check available GPU memory
        gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1e9  # Convert to GB
        print(f"Available GPU Memory: {gpu_memory:.1f} GB")

        # Add recommendation based on memory
        if gpu_memory < 4:
            print("Warning: Low GPU memory. Consider reducing batch size if you encounter OOM errors.")
    except Exception as e:
        print(f"Could not check GPU memory: {e}")
else:
    print("No GPU detected. Training will be much slower on CPU.")
    print("If you're using Colab, go to Runtime > Change runtime type and select GPU.")
```

```
Random seeds set to 42 for reproducible results
Available GPU Memory: 15.6 GB
```

Step 2: Choosing Your Dataset

You have several options for this exercise, depending on your computer's capabilities:

Option 1: MNIST (Basic - Works on Free Colab)

- Content: Handwritten digits (0-9)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if**: You're using free Colab or have a basic GPU

Option 2: Fashion-MNIST (Intermediate)

- Content: Clothing items (shirts, shoes, etc.)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if**: You want more interesting images but have limited GPU

Option 3: CIFAR-10 (Advanced)

- Content: Real-world objects (cars, animals, etc.)
- Image size: 32x32 pixels, Color (RGB)
- Training samples: 50,000
- Memory needed: ~4GB GPU
- Training time: ~1-2 hours on Colab
- **Choose this if**: You have Colab Pro or a good local GPU (8GB+ memory)

Option 4: CelebA (Expert)

- Content: Celebrity face images
- Image size: 64x64 pixels, Color (RGB)
- Training samples: 200,000
- Memory needed: ~8GB GPU
- Training time: ~3-4 hours on Colab
- **Choose this if**: You have excellent GPU (12GB+ memory)

To use your chosen dataset, uncomment its section in the code below and make sure all others are commented out.

```
#==============================================================================
# SECTION 2: DATASET SELECTION AND CONFIGURATION
#==============================================================================
# STUDENT INSTRUCTIONS:
# 1. Choose ONE dataset option based on your available GPU memory
# 2. Uncomment ONLY ONE dataset section below
# 3. Make sure all other dataset sections remain commented out

#-----------------------------------------
# OPTION 1: MNIST (Basic - 2GB GPU)
#-----------------------------------------
# Recommended for: Free Colab or basic GPU
# Memory needed: ~2GB GPU
# Training time: ~15-30 minutes

IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 64
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the MNIST dataset
# Hint: Use torchvision.datasets.MNIST with root='./data', train=True,
#       transform=transform, and download=True
# Then print a success message

# Enter your code here:
dataset = torchvision.datasets.MNIST(root='./data', train=True, transform=transform, download=True)
```

```
print(f"MNIST dataset loaded successfully! Total samples: {len(dataset)}")


#-------------------------------------------
# OPTION 2: Fashion-MNIST (Intermediate - 2GB GPU)
#-------------------------------------------
# Uncomment this section to use Fashion-MNIST instead
"""
IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 64
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the Fashion-MNIST dataset
# Hint: Very similar to MNIST but use torchvision.datasets.FashionMNIST

# Enter your code here:

"""

#-------------------------------------------
# OPTION 3: CIFAR-10 (Advanced - 4GB+ GPU)
#-------------------------------------------
# Uncomment this section to use CIFAR-10 instead
"""
IMG_SIZE = 32
IMG_CH = 3
N_CLASSES = 10
BATCH_SIZE = 32  # Reduced batch size for memory
EPOCHS = 50      # More epochs for complex data

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

dataset = torchvision.datasets.CIFAR10(root='./data', train=True, transform=transform, download=True)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
print(f"CIFAR-10 loaded successfully! Total samples: {len(dataset)}")
"""
```

```
MNIST dataset loaded successfully! Total samples: 60000
'\nIMG_SIZE = 32\nIMG_CH = 3\nN_CLASSES = 10\nBATCH_SIZE = 32  # Reduced batch size for memory\nEPOCHS = 50      # More epochs
for complex data\n\ntransform = transforms.Compose([\n    transforms.ToTensor(),\n    transforms.Normalize((0.5, 0.5, 0.5), (0.
5, 0.5, 0.5))\n])\n\ndataset = torchvision.datasets.CIFAR10(root=\'./data\', train=True, transform=transform, download=True)\nd
ataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)\nprint(f"CIFAR-10 loaded successfully! Total samples: {len
```

```
# Define the transformation (converting images to tensors)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

# Download and load the training data
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4, shuffle=True)

# Download and load the test data
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4, shuffle=False)

classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
print("CIFAR-10 is ready.")
```

```
CIFAR-10 is ready.
```

```
# Validating Dataset Selection
# Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:
```

```
# Validate dataset selection
if 'dataset' not in locals():
    raise ValueError("""
    ❌ ERROR: No dataset selected! Please uncomment exactly one dataset option.
    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    4. CelebA (Expert) - 8GB+ GPU
    """)

# Your code to validate GPU memory requirements
# Hint: Check torch.cuda.is_available() and use torch.cuda.get_device_properties(0).total_memory
# to get available GPU memory, then compare with dataset requirements

# Enter your code here:

# Heuristic GPU memory requirements *in bytes* based on IMG_SIZE / dataset choice
if IMG_SIZE == 28 and IMG_CH == 1 and N_CLASSES == 10:
    required_gb = 2  # MNIST or Fashion-MNIST
elif IMG_SIZE == 32 and IMG_CH == 3 and N_CLASSES == 10:
    required_gb = 4  # CIFAR-10
else:
    required_gb = 8  # Assume CelebA or larger custom dataset

required_bytes = required_gb * (1024 ** 3)

if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    total_mem_bytes = props.total_memory
    total_mem_gb = total_mem_bytes / (1024 ** 3)

    print(f"✅ GPU detected: {props.name}")
    print(f"   Total GPU memory: {total_mem_gb:.2f} GB")
    print(f"   Estimated required memory for this dataset: {required_gb} GB")

    if total_mem_bytes < required_bytes:
        print("⚠️ WARNING: Your GPU may not have enough memory for this dataset choice.")
        print("   You may experience out-of-memory errors. Consider using a smaller dataset option.")
    else:
        print("✅ Your GPU appears to have enough memory for this dataset choice.")
else:
    print("⚠️ WARNING: No GPU detected. Training will run on CPU and be much slower.")
    print("   For large datasets (like CIFAR-10 or CelebA), this may be very slow.")
```

```
✅ GPU detected: Tesla T4
   Total GPU memory: 14.56 GB
   Estimated required memory for this dataset: 2 GB
✅ Your GPU appears to have enough memory for this dataset choice.
```

```
#Dataset Properties and Data Loaders
#Now let's examine our dataset
#and set up the data loaders:

# Your code to check sample batch properties
sample_batch = next(iter(DataLoader(dataset, batch_size=1)))
images, labels = sample_batch
print(f"Dataset: {dataset.__class__.__name__}")
print(f"Total samples: {len(dataset)}")
print(f"Image shape: {images.shape}")
print(f"Image dtype: {images.dtype}")
print(f"Pixel value range: [{images.min():.2f}, {images.max():.2f}]")
print(f"Number of classes: {N_CLASSES}")


#==============================================================================
# SECTION 3: DATASET SPLITTING AND DATALOADER CONFIGURATION
#==============================================================================
# Create train-validation split

# Your code to create a train-validation split (80% train, 20% validation)
from torch.utils.data import random_split

train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size

train_dataset, val_dataset = random_split(
```

```
    dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

print(f"Training samples:   {len(train_dataset)}")
print(f"Validation samples: {len(val_dataset)}")


# Your code to create dataloaders for training and validation
num_workers = os.cpu_count() if os.cpu_count() is not None else 2

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,  num_workers=num_workers, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False, num_workers=num_workers, pin_memory=True)

print(f"Train batches:      {len(train_loader)}")
print(f"Validation batches: {len(val_loader)}")
```

```
Dataset: MNIST
Total samples: 60000
Image shape: torch.Size([1, 1, 28, 28])
Image dtype: torch.float32
Pixel value range: [-1.00, 1.00]
Number of classes: 10
Training samples:   48000
Validation samples: 12000
Train batches:      750
Validation batches: 188
```

Step 3: Building Our Model Components

Now we'll create the building blocks of our AI model. Think of these like LEGO pieces that we'll put together to make our number generator:

- GELUConvBlock: The basic building block that processes images
- DownBlock: Makes images smaller while finding important features
- UpBlock: Makes images bigger again while keeping the important features
- Other blocks: Help the model understand time and what number to generate

```
# Basic building block that processes images
class GELUConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, group_size):
        """
        Creates a block with convolution, normalization, and activation

        Args:
            in_ch (int): Number of input channels
            out_ch (int): Number of output channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Check that group_size is compatible with out_ch
        if out_ch % group_size != 0:
            print(f"Warning: out_ch ({out_ch}) is not divisible by group_size ({group_size})")
            # Adjust group_size to be compatible
            group_size = min(group_size, out_ch)
            while out_ch % group_size != 0:
                group_size -= 1
            print(f"Adjusted group_size to {group_size}")

        # Your code to create layers for the block
        self.model = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.GroupNorm(group_size, out_ch),
            nn.GELU()
        )

    def forward(self, x):
        # Your code for the forward pass
        return self.model(x)
```

```
# Rearranges pixels to downsample the image (2x reduction in spatial dimensions)
class RearrangePoolBlock(nn.Module):
```

```
    def __init__(self, in_chs, group_size):
        """
        Downsamples the spatial dimensions by 2x while preserving information

        Args:
            in_chs (int): Number of input channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Rearrange pixels: folds 2x2 spatial blocks into channels (4x more channels)
        self.rearrange = Rearrange('b c (h p1) (w p2) -> b (c p1 p2) h w', p1=2, p2=2)
        # Convolution to process the rearranged tensor (in_chs * 4 -> in_chs)
        self.conv = GELUConvBlock(in_chs * 4, in_chs, group_size)

    def forward(self, x):
        x = self.rearrange(x)
        x = self.conv(x)
        return x
```

```
#Let's implement the upsampling block for our U-Net architecture:
class DownBlock(nn.Module):
    """
    Downsampling block for encoding path in U-Net architecture.

    This block:
    1. Processes input features with two convolutional blocks
    2. Downsamples spatial dimensions by 2x using pixel rearrangement

    Args:
        in_chs (int): Number of input channels
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__()  # Simplified super() call, equivalent to original

        # Sequential processing of features
        layers = [
            GELUConvBlock(in_chs, out_chs, group_size),  # First conv block changes channel dimensions
            GELUConvBlock(out_chs, out_chs, group_size),  # Second conv block processes features
            RearrangePoolBlock(out_chs, group_size)       # Downsampling (spatial dims: H,W → H/2,W/2)
        ]
        self.model = nn.Sequential(*layers)

        # Log the configuration for debugging
        print(f"Created DownBlock: in_chs={in_chs}, out_chs={out_chs}, spatial_reduction=2x")

    def forward(self, x):
        """
        Forward pass through the DownBlock.

        Args:
            x (torch.Tensor): Input tensor of shape [B, in_chs, H, W]

        Returns:
            torch.Tensor: Output tensor of shape [B, out_chs, H/2, W/2]
        """
        return self.model(x)
```

```
# Now let's implement the upsampling block for our U-Net architecture:
class UpBlock(nn.Module):
    """
    Upsampling block for decoding path in U-Net architecture.

    This block:
    1. Takes features from the decoding path and corresponding skip connection
    2. Concatenates them along the channel dimension
    3. Upsamples spatial dimensions by 2x using transposed convolution
    4. Processes features through multiple convolutional blocks

    Args:
        in_chs_x (int): Number of input channels from the previous layer (x)
        in_chs_skip (int): Number of channels in the skip connection
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
```

```
    """
    def __init__(self, in_chs_x, in_chs_skip, out_chs, group_size):
        super().__init__()

        # Upsample `x` (from lower resolution) to match spatial dimensions of `skip`
        self.upsample_op = nn.ConvTranspose2d(in_chs_x, in_chs_x, kernel_size=2, stride=2)

        # Convolutional blocks after upsampling `x` and concatenating with `skip`
        # Input channels will be `in_chs_x` (after upsample) + `in_chs_skip`
        self.model_blocks = nn.Sequential(
            GELUConvBlock(in_chs_x + in_chs_skip, out_chs, group_size),
            GELUConvBlock(out_chs, out_chs, group_size)
        )

        print(f"Created UpBlock: x_in_chs={in_chs_x}, skip_in_chs={in_chs_skip}, out_chs={out_chs}, spatial_increase=2x")

    def forward(self, x, skip):
        """
        Forward pass through the UpBlock.

        Args:
            x (torch.Tensor): Input tensor from previous layer [B, in_chs_x, H, W]
            skip (torch.Tensor): Skip connection tensor from encoder [B, in_chs_skip, 2H, 2W]

        Returns:
            torch.Tensor: Output tensor with shape [B, out_chs, 2H, 2W]
        """
        print(f"DEBUG: UpBlock - INPUT x_shape: {x.shape}, skip_shape: {skip.shape}") # NEW
        # 1. Upsample x to double its spatial dimensions
        print(f"DEBUG: UpBlock - Before upsample x_shape: {x.shape}") # NEW
        x = self.upsample_op(x)
        print(f"DEBUG: UpBlock - After upsample x_shape: {x.shape}") # NEW

        # Handle potential spatial dimension mismatch from ConvTranspose2d (padding/cropping)
        target_h, target_w = skip.size()[2:]
        current_h, current_w = x.size()[2:]

        print(f"DEBUG: UpBlock - Before concat, after upsample/x_shape: {x.shape}, skip_shape: {skip.shape}")
        print(f"DEBUG: UpBlock - target_h={target_h}, current_h={current_h}")

        if current_h != target_h or current_w != target_w:
            print(f"DEBUG: UpBlock - Mismatch detected: x={current_h}x{current_w}, skip={target_h}x{target_w}. Applying adjustm
            diff_h = abs(current_h - target_h)
            diff_w = abs(current_w - target_w)
            if current_h > target_h: # Crop
                x = x[:, :, diff_h // 2 : current_h - (diff_h - diff_h // 2),
                           diff_w // 2 : current_w - (diff_w - diff_w // 2)]
            else: # Pad
                padding = [diff_w // 2, diff_w - diff_w // 2,
                           diff_h // 2, diff_h - diff_h // 2]
                x = F.pad(x, padding)
            print(f"DEBUG: UpBlock - After adjustment - x_shape: {x.shape}")

        print(f"DEBUG: UpBlock - IMMEDIATELY before concat: x_shape={x.shape}, skip_shape={skip.shape}") # NEW DEBUG PRINT
        # 2. Concatenate with the skip connection
        x = torch.cat([x, skip], dim=1)

        # 3. Pass through convolutional blocks
        x = self.model_blocks(x)
        return x


# Let's implement the downsampling block for our U-Net architecture:
class DownBlock(nn.Module):
    """
    Downsampling block for encoding path in U-Net architecture.

    This block:
    1. Processes input features with two convolutional blocks
    2. Downsamples spatial dimensions by 2x using pixel rearrangement

    Args:
        in_chs (int): Number of input channels
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
```

```
        super().__init__()

        # Sequential processing of features
        layers = [
            GELUConvBlock(in_chs, out_chs, group_size),   # First conv block changes channel dimensions
            GELUConvBlock(out_chs, out_chs, group_size),   # Second conv block processes features
            RearrangePoolBlock(out_chs, group_size)        # Downsampling (spatial dims: H,W → H/2,W/2)
        ]
        self.model = nn.Sequential(*layers)

        # Log the configuration for debugging
        print(f"Created DownBlock: in_chs={in_chs}, out_chs={out_chs}, spatial_reduction=2x")

    def forward(self, x):
        """
        Forward pass through the DownBlock.

        Args:
            x (torch.Tensor): Input tensor of shape [B, in_chs, H, W]

        Returns:
            torch.Tensor: Output tensor of shape [B, out_chs, H/2, W/2]
        """
        return self.model(x)
```

```
# Here we implement the time embedding block for our U-Net architecture:
# Helps the model understand time steps in diffusion process
class SinusoidalPositionEmbedBlock(nn.Module):
    """
    Creates sinusoidal embeddings for time steps in diffusion process.

    This embedding scheme is adapted from the Transformer architecture and
    provides a unique representation for each time step that preserves
    relative distance information.

    Args:
        dim (int): Embedding dimension
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, time):
        """
        Computes sinusoidal embeddings for given time steps.

        Args:
            time (torch.Tensor): Time steps tensor of shape [batch_size]

        Returns:
            torch.Tensor: Time embeddings of shape [batch_size, dim]
        """
        device = time.device
        half_dim = self.dim // 2
        embeddings = torch.log(torch.tensor(10000.0, device=device)) / (half_dim - 1)
        embeddings = torch.exp(torch.arange(half_dim, device=device) * -embeddings)
        embeddings = time[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings
```

```
# Helps the model understand which number/image to draw (class conditioning)
class EmbedBlock(nn.Module):
    """
    Creates embeddings for class conditioning in diffusion models.

    This module transforms a one-hot or index representation of a class
    into a rich embedding that can be added to feature maps.

    Args:
        input_dim (int): Input dimension (typically number of classes)
        emb_dim (int): Output embedding dimension
    """
    def __init__(self, input_dim, emb_dim):
        super(EmbedBlock, self).__init__()
```

```
        self.input_dim = input_dim

        # Embedding layers: Linear -> GELU -> Linear -> Unflatten for broadcasting
        self.model = nn.Sequential(
            nn.Linear(input_dim, emb_dim),
            nn.GELU(),
            nn.Linear(emb_dim, emb_dim),
            nn.Unflatten(1, (emb_dim, 1, 1))  # Reshape to [B, emb_dim, 1, 1] for feature map broadcasting
        )

    def forward(self, x):
        """
        Computes class embeddings for the given class indices.

        Args:
            x (torch.Tensor): Class indices or one-hot encodings [batch_size, input_dim]

        Returns:
            torch.Tensor: Class embeddings of shape [batch_size, emb_dim, 1, 1]
                          (ready to be added to feature maps)
        """
        x = x.view(-1, self.input_dim)
        return self.model(x)
```

```
# Main U-Net model that puts everything together
class UNet(nn.Module):
    """
    U-Net architecture for diffusion models with time and class conditioning.

    This architecture follows the standard U-Net design with:
    1. Downsampling path that reduces spatial dimensions
    2. Middle processing blocks
    3. Upsampling path that reconstructs spatial dimensions
    4. Skip connections between symmetric layers

    The model is conditioned on:
    - Time step (where we are in the diffusion process)
    - Class labels (what we want to generate)

    Args:
        T (int): Number of diffusion time steps
        img_ch (int): Number of image channels
        img_size (int): Size of input images
        down_chs (list): Channel dimensions for each level of U-Net
        t_embed_dim (int): Dimension for time embeddings
        c_embed_dim (int): Dimension for class embeddings
    """
    def __init__(self, T, img_ch, img_size, down_chs, t_embed_dim, c_embed_dim):
        super().__init__()

        group_size = 8  # GroupNorm group size used throughout

        # Time embedding: sinusoidal -> linear -> GELU -> linear
        self.time_embed = nn.Sequential(
            SinusoidalPositionEmbedBlock(t_embed_dim),
            nn.Linear(t_embed_dim, t_embed_dim),
            nn.GELU(),
            nn.Linear(t_embed_dim, t_embed_dim),
            nn.Unflatten(1, (t_embed_dim, 1, 1))  # [B, t_embed_dim, 1, 1] for broadcasting
        )

        # Class embedding
        # c_embed_dim is N_CLASSES, down_chs[-1] is the output dim of the embedding (bottleneck channels)
        self.class_embed = EmbedBlock(c_embed_dim, down_chs[-1])

        # Initial convolution: img_ch -> down_chs[0]
        self.init_conv = GELUConvBlock(img_ch, down_chs[0], group_size)

        # Downsampling path
        self.down_blocks = nn.ModuleList([
            DownBlock(down_chs[i], down_chs[i + 1], group_size)
            for i in range(len(down_chs) - 1)
        ])

        # Middle blocks (bottleneck)
        self.mid_block1 = GELUConvBlock(down_chs[-1], down_chs[-1], group_size)
```

```
        self.mid_block2 = GELUConvBlock(down_chs[-1], down_chs[-1], group_size)

        # Upsampling path (reverse channel order)
        # up_chs: (128, 64, 32)
        # down_chs: (32, 64, 128)
        self.up_blocks = nn.ModuleList()
        up_chs_list = list(reversed(down_chs)) # e.g., (128, 64, 32)

        for i in range(len(up_chs_list) - 1): # i = 0, 1
            in_ch_x = up_chs_list[i] # Current `x` channels (e.g., 128 then 64)
            out_ch = up_chs_list[i+1] # Output channels for this upsampling level (e.g., 64 then 32)
            # The skip connection channels correspond to the original downsampling levels
            # For up_blocks[0] (x from H/4 to H/2), skip should be from down_chs[1] (64 channels at H/2)
            # For up_blocks[1] (x from H/2 to H), skip should be from down_chs[0] (32 channels at H)
            in_ch_skip = down_chs[len(down_chs) - 2 - i] # Correctly map skip channels
            self.up_blocks.append(UpBlock(in_ch_x, in_ch_skip, out_ch, group_size))

        # Final convolution: down_chs[0] -> img_ch
        self.final_conv = nn.Conv2d(down_chs[0], img_ch, kernel_size=1)

        print(f"Created UNet with {len(down_chs)} scale levels")
        print(f"Channel dimensions: {down_chs}")

    def forward(self, x, t, c, c_mask):
        """
        Forward pass through the UNet.

        Args:
            x (torch.Tensor): Input noisy image [B, img_ch, H, W]
            t (torch.Tensor): Diffusion time steps [B]
            c (torch.Tensor): Class labels [B, c_embed_dim]
            c_mask (torch.Tensor): Mask for conditional generation [B, 1]

        Returns:
            torch.Tensor: Predicted noise in the input image [B, img_ch, H, W]
        """
        # Time embedding [B, t_embed_dim, 1, 1]
        t_emb = self.time_embed(t)

        # Class embedding with classifier-free guidance mask [B, down_chs[-1], 1, 1]
        c_emb = self.class_embed(c) * c_mask.view(-1, 1, 1, 1)

        # Initial feature extraction
        x = self.init_conv(x)

        # Downsampling path — store outputs for skip connections
        skip_connections = []
        for down in self.down_blocks:
            skip_connections.append(x) # Store x *before* it gets downsampled by the current 'down' block
            x = down(x)

        # Middle processing + add time and class conditioning
        x = self.mid_block1(x)
        x = x + t_emb + c_emb
        x = self.mid_block2(x)

        # Upsampling path with skip connections
        # The up_blocks are already ModuleList, iterate and call
        for up_block in self.up_blocks:
            skip = skip_connections.pop() # Retrieve corresponding skip connection
            x = up_block(x, skip)

        # Final projection back to image space
        x = self.final_conv(x)
        return x
```

Step 4: Setting Up The Diffusion Process

Now we'll create the process of adding and removing noise from images. Think of it like:

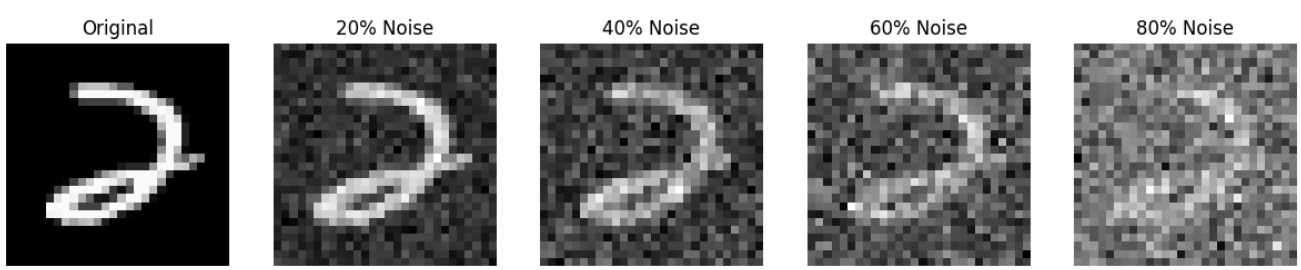
1. Adding fog: Slowly making the image more and more blurry until you can't see it
2. Removing fog: Teaching the AI to gradually make the image clearer
3. Controlling the process: Making sure we can generate specific numbers we want

```
# Set up the noise schedule
n_steps = 100  # How many steps to go from clear image to noise
beta_start = 0.0001  # Starting noise level (small)
beta_end = 0.02      # Ending noise level (larger)

# Create schedule of gradually increasing noise levels
beta = torch.linspace(beta_start, beta_end, n_steps).to(device)

# Calculate important values used in diffusion equations
alpha = 1 - beta  # Portion of original image to keep at each step
alpha_bar = torch.cumprod(alpha, dim=0)  # Cumulative product of alphas
sqrt_alpha_bar = torch.sqrt(alpha_bar)  # For scaling the original image
sqrt_one_minus_alpha_bar = torch.sqrt(1 - alpha_bar)  # For scaling the noise
```

```
# Function to add noise to images (forward diffusion process)
def add_noise(x_0, t):
    """
    Add noise to images according to the forward diffusion process.

    The formula is: x_t = √(α_bar_t) * x_0 + √(1-α_bar_t) * ε
    where ε is random noise and α_bar_t is the cumulative product of (1-β).

    Args:
        x_0 (torch.Tensor): Original clean image [B, C, H, W]
        t (torch.Tensor): Timestep indices indicating noise level [B]

    Returns:
        tuple: (noisy_image, noise_added)
            - noisy_image is the image with noise added
            - noise_added is the actual noise that was added (for training)
    """
    # Create random Gaussian noise with same shape as image
    noise = torch.randn_like(x_0)

    # Get noise schedule values for the specified timesteps
    # Reshape to allow broadcasting with image dimensions
    sqrt_alpha_bar_t = sqrt_alpha_bar[t].reshape(-1, 1, 1, 1)
    sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1)

    # Apply the forward diffusion equation:
    # x_t = √(α_bar_t) * x_0 + √(1-α_bar_t) * ε
    x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise

    return x_t, noise
```

```
@torch.no_grad()
def remove_noise(x_t, t, model, c, c_mask):
    """
    Remove noise from images using the learned reverse diffusion process.
    """
    # Predict the noise in the image using our model
    predicted_noise = model(x_t, t, c, c_mask)

    # Get noise schedule values for the current timestep
    alpha_t          = alphas[t].reshape(-1, 1, 1, 1)
    alpha_bar_t      = alpha_bar[t].reshape(-1, 1, 1, 1)
    beta_t           = betas[t].reshape(-1, 1, 1, 1)
    sqrt_one_minus_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1)

    # Special case: if we're at the first timestep (t=0), we're done
    if t[0] == 0:
        return x_t

    # Calculate the mean of the denoised distribution
    mean = (1 / torch.sqrt(alpha_t)) * (
        x_t - (beta_t / sqrt_one_minus_t) * predicted_noise
    )

    # Add a small amount of random noise
    noise = torch.randn_like(x_t)

    return mean + torch.sqrt(beta_t) * noise
```

```
# Visualization function to show how noise progressively affects images
def show_noise_progression(image, num_steps=5):
    """
    Visualize how an image gets progressively noisier in the diffusion process.

    Args:
        image (torch.Tensor): Original clean image [C, H, W]
        num_steps (int): Number of noise levels to show
    """
    plt.figure(figsize=(15, 3))

    # Show original image
    plt.subplot(1, num_steps, 1)
    if IMG_CH == 1:  # Grayscale image
        plt.imshow(image[0].cpu(), cmap='gray')
    else:  # Color image
        img = image.permute(1, 2, 0).cpu()  # Change from [C,H,W] to [H,W,C]
        if img.min() < 0:  # If normalized between -1 and 1
            img = (img + 1) / 2  # Rescale to [0,1] for display
        plt.imshow(img)
    plt.title('Original')
    plt.axis('off')

    # Show progressively noisier versions
    for i in range(1, num_steps):
        # Calculate timestep index based on percentage through the process
        t_idx = int((i/num_steps) * n_steps)
        t = torch.tensor([t_idx]).to(device)

        # Add noise corresponding to timestep t
        noisy_image, _ = add_noise(image.unsqueeze(0), t)

        # Display the noisy image
        plt.subplot(1, num_steps, i+1)
        if IMG_CH == 1:
            plt.imshow(noisy_image[0][0].cpu(), cmap='gray')
        else:
            img = noisy_image[0].permute(1, 2, 0).cpu()
            if img.min() < 0:
                img = (img + 1) / 2
            plt.imshow(img)
        plt.title(f'{int((i/num_steps) * 100)}% Noise')
        plt.axis('off')
    plt.show()

# Show an example of noise progression on a real image
sample_batch = next(iter(train_loader))  # Get first batch
sample_image = sample_batch[0][0].to(device)  # Get first image
show_noise_progression(sample_image)

# Student Activity: Try different noise schedules
# Uncomment and modify these lines to experiment:
"""
# Try a non-linear noise schedule
beta_alt = torch.linspace(beta_start, beta_end, n_steps)**2
alpha_alt = 1 - beta_alt
alpha_bar_alt = torch.cumprod(alpha_alt, dim=0)
# How would this affect the diffusion process?
"""
```




```
'\n# Try a non-linear noise schedule\nbeta_alt = torch.linspace(beta_start, beta_end, n_steps)**2\nalpha_alt = 1 - beta_alt\nal
pha_bar_alt = torch.cumprod(alpha_alt, dim=0)\n# How would this affect the diffusion process?\n'
```

Step 5: Training Our Model

Now we'll teach our AI to generate images. This process:

1. Takes a clear image
2. Adds random noise to it
3. Asks our AI to predict what noise was added
4. Helps our AI learn from its mistakes

This will take a while, but we'll see progress as it learns!

```
# Create our model and move it to GPU if available
model = UNet(
    T=n_steps,                 # Number of diffusion time steps
    img_ch=IMG_CH,             # Number of channels in our images (1 for grayscale, 3 for RGB)
    img_size=IMG_SIZE,         # Size of input images (28 for MNIST, 32 for CIFAR-10)
    down_chs=(32, 64, 128),    # Channel dimensions for each level of U-Net
    t_embed_dim=128,             # Dimension for time step embeddings (was 8, changed to 128 to match down_chs[-1])
    c_embed_dim=N_CLASSES      # Number of classes for conditioning
).to(device)

# Print model summary
print(f"\n{'='*50}")
print(f"MODEL ARCHITECTURE SUMMARY")
print(f"{'='*50}")
print(f"Input resolution: {IMG_SIZE}x{IMG_SIZE}")
print(f"Input channels: {IMG_CH}")
print(f"Time steps: {n_steps}")
print(f"Condition classes: {N_CLASSES}")
print(f"GPU acceleration: {'Yes' if device.type == 'cuda' else 'No'}")

# Validate model parameters and estimate memory requirements
def count_parameters(model):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return total, trainable

def estimate_memory_mb(model):
    total_params, _ = count_parameters(model)
    # Each parameter is a float32 (4 bytes), convert to MB
    return (total_params * 4) / (1024 ** 2)

total_params, trainable_params = count_parameters(model)
estimated_mem = estimate_memory_mb(model)

print(f"\nTotal parameters:     {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
print(f"Estimated model size: {estimated_mem:.2f} MB")

# Your code to verify data ranges and integrity
def check_data_ranges(loader, name="Dataset", num_batches=3):
    all_mins, all_maxs, all_means = [], [], []
    for i, (imgs, _) in enumerate(loader):
        if i >= num_batches:
            break
        all_mins.append(imgs.min().item())
        all_maxs.append(imgs.max().item())
        all_means.append(imgs.mean().item())
    print(f"\n{name} data range check (first {num_batches} batches):")
    print(f"  Min:  {min(all_mins):.4f}")
    print(f"  Max:  {max(all_maxs):.4f}")
    print(f"  Mean: {sum(all_means)/len(all_means):.4f}")
    if min(all_mins) >= -1.1 and max(all_maxs) <= 1.1:
        print(f"  ✅ Values are in expected range [-1, 1]")
    else:
        print(f"  ⚠️ WARNING: Values may be outside expected range [-1, 1]")

check_data_ranges(train_loader, name="Training")
check_data_ranges(val_loader,   name="Validation")


# Set up the optimizer with parameters tuned for diffusion models
# Note: Lower learning rates tend to work better for diffusion models
initial_lr = 0.001  # Starting learning rate
```

```
weight_decay = 1e-5  # L2 regularization to prevent overfitting

optimizer = Adam(
    model.parameters(),
    lr=initial_lr,
    weight_decay=weight_decay
)

# Learning rate scheduler to reduce LR when validation loss plateaus
# This helps fine-tune the model toward the end of training
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',              # Reduce LR when monitored value stops decreasing
    factor=0.5,              # Multiply LR by this factor
    patience=5,              # Number of epochs with no improvement after which LR will be reduced
    # verbose=True,            # Print message when LR is reduced
    min_lr=1e-6              # Lower bound on the learning rate
)

# STUDENT EXPERIMENT:
# Try different channel configurations and see how they affect:
# 1. Model size (parameter count)
# 2. Training time
# 3. Generated image quality
#
# Suggestions:
# - Smaller: down_chs=(16, 32, 64)
# - Larger: down_chs=(64, 128, 256, 512)
```

```
Created DownBlock: in_chs=32, out_chs=64, spatial_reduction=2x
Created DownBlock: in_chs=64, out_chs=128, spatial_reduction=2x
Created UpBlock: x_in_chs=128, skip_in_chs=64, out_chs=64, spatial_increase=2x
Created UpBlock: x_in_chs=64, skip_in_chs=32, out_chs=32, spatial_increase=2x
Created UNet with 3 scale levels
Channel dimensions: (32, 64, 128)

==================================================
MODEL ARCHITECTURE SUMMARY
==================================================
Input resolution: 28x28
Input channels: 1
Time steps: 100
Condition classes: 10
GPU acceleration: Yes

Total parameters:     1,629,537
Trainable parameters: 1,629,537
Estimated model size: 6.22 MB

Training data range check (first 3 batches):
  Min:  -1.0000
  Max:  1.0000
  Mean: -0.7462
  ✅ Values are in expected range [-1, 1]

Validation data range check (first 3 batches):
  Min:  -1.0000
  Max:  1.0000
  Mean: -0.7449
  ✅ Values are in expected range [-1, 1]
```

```
# Define helper functions needed for training and evaluation
def validate_model_parameters(model):
    """
    Counts model parameters and estimates memory usage.
    """
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

    print(f"Total parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")

    # Estimate memory requirements (very approximate)
    param_memory = total_params * 4 / (1024 ** 2)  # MB for params (float32)
    grad_memory = trainable_params * 4 / (1024 ** 2)  # MB for gradients
    buffer_memory = param_memory * 2  # Optimizer state, forward activations, etc.

    print(f"Estimated GPU memory usage: {param_memory + grad_memory + buffer_memory:.1f} MB")
```

```python
# Define helper functions for verifying data ranges
def verify_data_range(dataloader, name="Dataset"):
    """
    Verifies the range and integrity of the data.
    """
    batch = next(iter(dataloader))[0]
    print(f"\n{name} range check:")
    print(f"Shape: {batch.shape}")
    print(f"Data type: {batch.dtype}")
    print(f"Min value: {batch.min().item():.2f}")
    print(f"Max value: {batch.max().item():.2f}")
    print(f"Contains NaN: {torch.isnan(batch).any().item()}")
    print(f"Contains Inf: {torch.isinf(batch).any().item()}")

# Define helper functions for generating samples during training
def generate_samples(model, n_samples=10):
    """
    Generates sample images using the model for visualization during training.
    """
    model.eval()
    with torch.no_grad():
        # Generate digits 0-9 for visualization
        samples = []
        for digit in range(min(n_samples, 10)):
            # Start with random noise
            x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

            # Set up conditioning for the digit
            c = torch.tensor([digit]).to(device)
            c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
            c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

            # Remove noise step by step
            for t in range(n_steps-1, -1, -1):
                t_batch = torch.full((1,), t).to(device)
                x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

            samples.append(x)

        # Combine samples and display
        samples = torch.cat(samples, dim=0)
        grid = make_grid(samples, nrow=min(n_samples, 5), normalize=True)

        plt.figure(figsize=(10, 4))

        # Display based on channel configuration
        if IMG_CH == 1:
            plt.imshow(grid[0].cpu(), cmap='gray')
        else:
            plt.imshow(grid.permute(1, 2, 0).cpu())

        plt.axis('off')
        plt.title('Generated Samples')
        plt.show()

# Define helper functions for safely saving models
def safe_save_model(model, path, optimizer=None, epoch=None, best_loss=None):
    """
    Safely saves model with error handling and backup.
    """
    try:
        # Create a dictionary with all the elements to save
        save_dict = {
            'model_state_dict': model.state_dict(),
        }

        # Add optional elements if provided
        if optimizer is not None:
            save_dict['optimizer_state_dict'] = optimizer.state_dict()
        if epoch is not None:
            save_dict['epoch'] = epoch
        if best_loss is not None:
            save_dict['best_loss'] = best_loss

        # Create a backup of previous checkpoint if it exists
        if os.path.exists(path):
```

```
            backup_path = path + '.backup'
            try:
                os.replace(path, backup_path)
                print(f"Created backup at {backup_path}")
            except Exception as e:
                print(f"Warning: Could not create backup - {e}")

        # Save the new checkpoint
        torch.save(save_dict, path)
        print(f"Model successfully saved to {path}")

    except Exception as e:
        print(f"Error saving model: {e}")
        print("Attempting emergency save...")

        try:
            emergency_path = path + '.emergency'
            torch.save(model.state_dict(), emergency_path)
            print(f"Emergency save successful: {emergency_path}")
        except:
            print("Emergency save failed. Could not save model.")
```

```
# Implementation of the training step function
def train_step(x, c):
    """
    Performs a single training step for the diffusion model.

    This function:
    1. Prepares class conditioning
    2. Samples random timesteps for each image
    3. Adds corresponding noise to the images
    4. Asks the model to predict the noise
    5. Calculates the loss between predicted and actual noise

    Args:
        x (torch.Tensor): Batch of clean images [batch_size, channels, height, width]
        c (torch.Tensor): Batch of class labels [batch_size]

    Returns:
        torch.Tensor: Mean squared error loss value
    """
    # Convert number labels to one-hot encoding for class conditioning
    c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)

    # Create conditioning mask (all ones for standard training)
    c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

    # Pick random timesteps for each image in the batch
    t = torch.randint(0, n_steps, (x.shape[0],)).to(device)

    # Add noise to images according to the forward diffusion process
    x_t, noise = add_noise(x, t)

    # The model tries to predict the exact noise that was added
    predicted_noise = model(x_t, t, c_one_hot, c_mask)

    # Calculate MSE loss between predicted and actual noise
    loss = F.mse_loss(predicted_noise, noise)

    return loss
```

```
#==============================================================================
# NOISE SCHEDULE CONFIGURATION
#==============================================================================
n_steps = 1000  # Number of diffusion steps
beta_start = 0.0001
beta_end = 0.02

# Create a linear beta schedule
betas = torch.linspace(beta_start, beta_end, n_steps).to(device)

# Pre-calculate values for the forward diffusion process
alphas = 1. - betas
alpha_bar = torch.cumprod(alphas, dim=0)
sqrt_alpha_bar = torch.sqrt(alpha_bar)
sqrt_one_minus_alpha_bar = torch.sqrt(1. - alpha_bar)
```

```
print(f"Noise schedule initialized for {n_steps} steps.")
```

```
Noise schedule initialized for 1000 steps.
```

```
# Implementation of the main training loop
# Training configuration
early_stopping_patience = 10  # Number of epochs without improvement before stopping
gradient_clip_value = 1.0     # Maximum gradient norm for stability
display_frequency = 100       # How often to show progress (in steps)
generate_frequency = 500      # How often to generate samples (in steps)

# Progress tracking variables
best_loss = float('inf')
train_losses = []
val_losses = []
no_improve_epochs = 0

# Training loop
print("\n" + "="*50)
print("STARTING TRAINING")
print("="*50)
model.train()

try:
    for epoch in range(EPOCHS):
        print(f"\nEpoch {epoch+1}/{EPOCHS}")
        print("-" * 20)

        # Training phase
        model.train()
        epoch_losses = []

        # Process each batch
        for step, (images, labels) in enumerate(train_loader):
            images = images.to(device)
            labels = labels.to(device)

            # Training step
            optimizer.zero_grad()
            loss = train_step(images, labels)
            loss.backward()

            # Add gradient clipping for stability
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=gradient_clip_value)

            optimizer.step()
            epoch_losses.append(loss.item())

            # Show progress at regular intervals
            if step % display_frequency == 0:
                print(f"  Step {step}/{len(train_loader)}, Loss: {loss.item():.4f}")

                # Generate samples less frequently to save time
                if step % generate_frequency == 0 and step > 0:
                    print("  Generating samples...")
                    generate_samples(model, n_samples=5)

        # End of epoch - calculate average training loss
        avg_train_loss = sum(epoch_losses) / len(epoch_losses)
        train_losses.append(avg_train_loss)
        print(f"\nTraining - Epoch {epoch+1} average loss: {avg_train_loss:.4f}")

        # Validation phase
        model.eval()
        val_epoch_losses = []
        print("Running validation...")

        with torch.no_grad():  # Disable gradients for validation
            for val_images, val_labels in val_loader:
                val_images = val_images.to(device)
                val_labels = val_labels.to(device)

                # Calculate validation loss
                val_loss = train_step(val_images, val_labels)
                val_epoch_losses.append(val_loss.item())
```

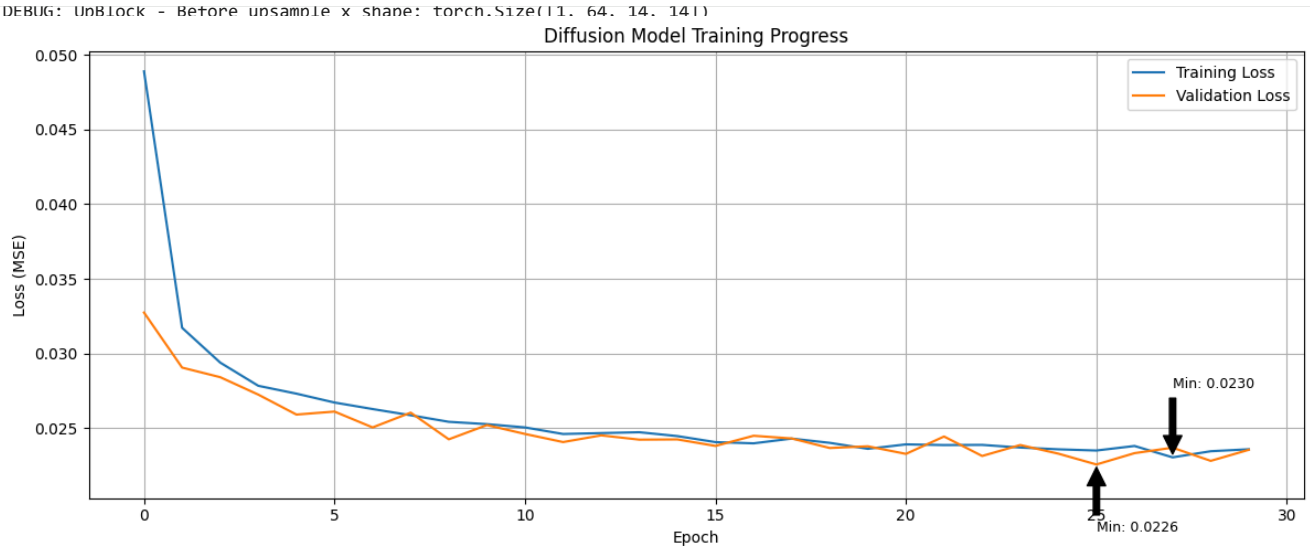
```
        # Calculate average validation loss
        avg_val_loss = sum(val_epoch_losses) / len(val_epoch_losses)
        val_losses.append(avg_val_loss)
        print(f"Validation - Epoch {epoch+1} average loss: {avg_val_loss:.4f}")

        # Learning rate scheduling based on validation loss
        scheduler.step(avg_val_loss)
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Learning rate: {current_lr:.6f}")

        # Generate samples at the end of each epoch
        if epoch % 2 == 0 or epoch == EPOCHS - 1:
            print("\nGenerating samples for visual progress check...")
            generate_samples(model, n_samples=10)

        # Save best model based on validation loss
        if avg_val_loss < best_loss:
            best_loss = avg_val_loss
            safe_save_model(model, 'best_diffusion_model.pt', optimizer, epoch, best_loss)
            print(f"✓ New best model saved! (Val Loss: {best_loss:.4f})")
            no_improve_epochs = 0
        else:
            no_improve_epochs += 1
            print(f"No improvement for {no_improve_epochs}/{early_stopping_patience} epochs")

        # Early stopping
        if no_improve_epochs >= early_stopping_patience:
            print("\nEarly stopping triggered! No improvement in validation loss.")
            break

        # Plot loss curves every few epochs
        if epoch % 5 == 0 or epoch == EPOCHS - 1:
            plt.figure(figsize=(10, 5))
            plt.plot(train_losses, label='Training Loss')
            plt.plot(val_losses, label='Validation Loss')
            plt.xlabel('Epoch')
            plt.ylabel('Loss')
            plt.title('Training and Validation Loss')
            plt.legend()
            plt.grid(True)
            plt.show()

    except KeyboardInterrupt:
        print("\n⚠️ Training interrupted by user.")
    except Exception as e:
        print(f"\n❌ Training error: {e}")
        raise


    # Final wrap-up
    print("\n" + "="*50)
    print("TRAINING COMPLETE")
    print("="*50)
    print(f"Best validation loss: {best_loss:.4f}")

    # Generate final samples
    print("Generating final samples...")
    generate_samples(model, n_samples=10)

    # Display final loss curves
    plt.figure(figsize=(12, 5))
    plt.plot(train_losses, label='Training Loss')
    plt.plot(val_losses, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid(True)
    plt.show()

    # Clean up memory
    torch.cuda.empty_cache()
```

```
Streaming output truncated to the last 5000 lines.
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
```

```
# Plot training progress
plt.figure(figsize=(12, 5))

# Plot training and validation losses for comparison
plt.plot(train_losses, label='Training Loss')
if len(val_losses) > 0:  # Only plot validation if it exists
    plt.plot(val_losses, label='Validation Loss')

# Improve the plot with better labels and styling
plt.title('Diffusion Model Training Progress')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True)

# Add annotations for key points
if len(train_losses) > 1:
    min_train_idx = train_losses.index(min(train_losses))
    plt.annotate(f'Min: {min(train_losses):.4f}',
                 xy=(min_train_idx, min(train_losses)),
                 xytext=(min_train_idx, min(train_losses)*1.2),
                 arrowprops=dict(facecolor='black', shrink=0.05),
                 fontsize=9)
```

```
# Add validation min point if available
if len(val_losses) > 1:
    min_val_idx = val_losses.index(min(val_losses))
    plt.annotate(f'Min: {min(val_losses):.4f}',
                 xy=(min_val_idx, min(val_losses)),
                 xytext=(min_val_idx, min(val_losses)*0.8),
                 arrowprops=dict(facecolor='black', shrink=0.05),
                 fontsize=9)

# Set y-axis to start from 0 or slightly lower than min value
plt.ylim(bottom=max(0, min(min(train_losses) if train_losses else float('inf'),
                           min(val_losses) if val_losses else float('inf'))*0.9))

plt.tight_layout()
plt.show()

# Add statistics summary for students to analyze
print("\nTraining Statistics:")
print("-" * 30)
if train_losses:
    print(f"Starting training loss:    {train_losses[0]:.4f}")
    print(f"Final training loss:       {train_losses[-1]:.4f}")
    print(f"Best training loss:        {min(train_losses):.4f}")
    print(f"Training loss improvement: {((train_losses[0] - min(train_losses)) / train_losses[0] * 100):.1f}%")

if val_losses:
    print("\nValidation Statistics:")
    print("-" * 30)
    print(f"Starting validation loss: {val_losses[0]:.4f}")
    print(f"Final validation loss:    {val_losses[-1]:.4f}")
    print(f"Best validation loss:     {min(val_losses):.4f}")

# STUDENT EXERCISE:
# 1. Try modifying this plot to show a smoothed version of the losses
# 2. Create a second plot showing the ratio of validation to training loss
#    (which can indicate overfitting when the ratio increases)
```

```
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
```

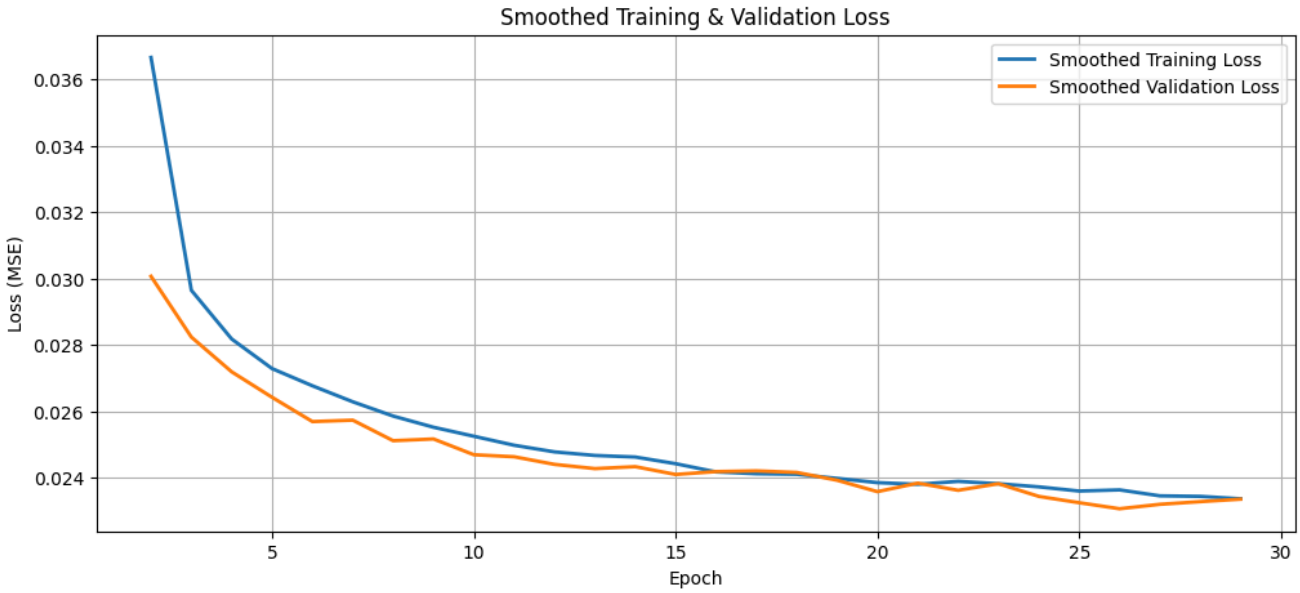



```
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
```

```
# ================================
# EXERCISE 1: Smoothed Loss Curves
# ================================
```

```
def smooth(values, window=3):
    """Simple moving average for smoothing curves."""
    if len(values) < window:
        return values
    return [sum(values[i-window:i]) / window for i in range(window, len(values)+1)]

plt.figure(figsize=(12, 5))
plt.title("Smoothed Training & Validation Loss")

smoothing_window = 3 # Define the window size for plotting the x-axis

# Smoothed curves
smooth_train = smooth(train_losses, window=smoothing_window)
# The x-axis should start from `smoothing_window - 1` (0-indexed epoch)
# and end at `len(train_losses) - 1` (last 0-indexed epoch).
# This range correctly produces `len(train_losses) - (smoothing_window - 1)` elements.
plt.plot(range(smoothing_window - 1, len(train_losses)), smooth_train, label='Smoothed Training Loss', linewidth=2)

if len(val_losses) > 0:
    smooth_val = smooth(val_losses, window=smoothing_window)
    plt.plot(range(smoothing_window - 1, len(val_losses)), smooth_val, label='Smoothed Validation Loss', linewidth=2)

plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.grid(True)
plt.show()
```

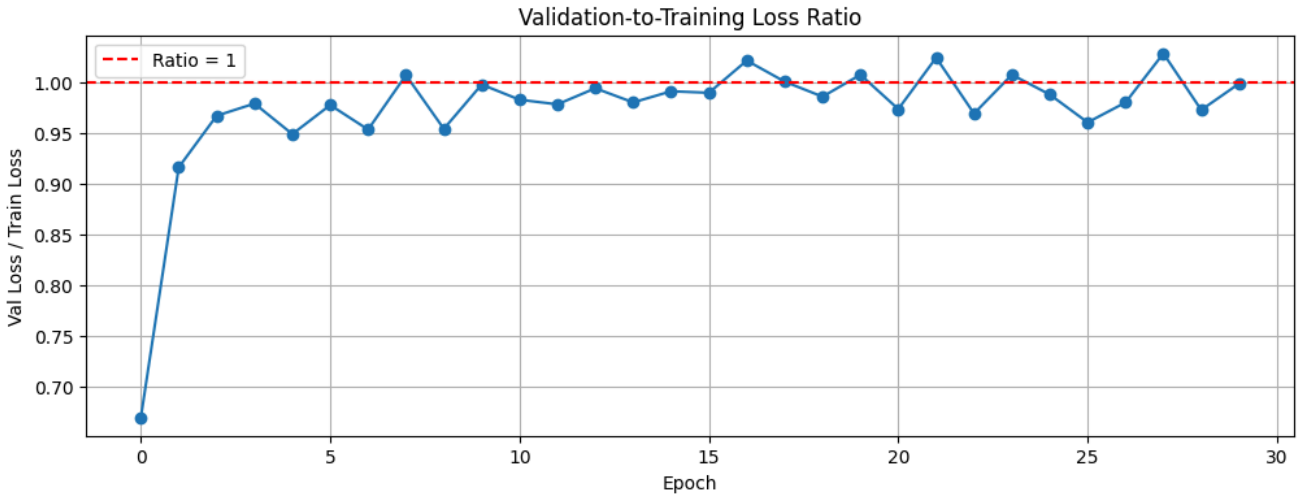
```
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
```




```
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
```

```
# ==========================================
# EXERCISE 2: Validation/Training Loss Ratio
# ==========================================

if len(val_losses) == 0:
    print("No validation losses available; cannot compute ratio.")
else:
    ratio = [v / t for v, t in zip(val_losses, train_losses)]

    plt.figure(figsize=(12, 4))
    plt.title("Validation-to-Training Loss Ratio")
    plt.plot(ratio, marker='o')
    plt.xlabel("Epoch")
    plt.ylabel("Val Loss / Train Loss")
    plt.grid(True)
    plt.axhline(1.0, color='red', linestyle='--', label='Ratio = 1')
    plt.legend()
    plt.show()

    # Interpretation
    print("\nInterpretation Guide:")
    print("- Ratio close to 1: good generalization")
```

```
print("- Ratio > 1.3: model may be overfitting")
print("- Ratio decreasing: model improving")
```




```
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
```

Step 6: Generating New Images

Now that our model is trained, let's generate some new images! We can:

1. Generate specific numbers
2. Generate multiple versions of each number
3. See how the generation process works step by step

```
def generate_number(model, number, n_samples=4):
    """
    Generate multiple versions of a specific number using the diffusion model.

    Args:
        model (nn.Module): The trained diffusion model
        number (int): The digit to generate (0-9)
        n_samples (int): Number of variations to generate

    Returns:
        torch.Tensor: Generated images of shape [n_samples, IMG_CH, IMG_SIZE, IMG_SIZE]
    """
    model.eval()  # Set model to evaluation mode
    with torch.no_grad():  # No need for gradients during generation
        # Start with random noise
        samples = torch.randn(n_samples, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

        # Set up the number we want to generate
        c = torch.full((n_samples,), number).to(device)
        c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
        # Correctly sized conditioning mask
        c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

        # Display progress information
        print(f"Generating {n_samples} versions of number {number}...")

        # Remove noise step by step
        for t in range(n_steps-1, -1, -1):
            t_batch = torch.full((n_samples,), t).to(device)
            samples = remove_noise(samples, t_batch, model, c_one_hot, c_mask)

            # Optional: Display occasional progress updates
            if t % (n_steps // 5) == 0:
                print(f"  Denoising step {n_steps-1-t}/{n_steps-1} completed")

        return samples
```

```
# Generate 4 versions of each number
plt.figure(figsize=(20, 10))
for i in range(10):
    # Generate samples for current digit
    samples = generate_number(model, i, n_samples=4)

    # Display each sample
    for j in range(4):
        # Use 2 rows, 10 digits per row, 4 samples per digit
        # i//5 determines the row (0 or 1)
        # i%5 determines the position in the row (0-4)
        # j is the sample index within each digit (0-3)
        plt.subplot(5, 8, (i%5)*8 + (i//5)*4 + j + 1)

        # Display the image correctly based on channel configuration
        if IMG_CH == 1:  # Grayscale
            plt.imshow(samples[j][0].cpu(), cmap='gray')
        else:  # Color image
            img = samples[j].permute(1, 2, 0).cpu()
            # Rescale from [-1, 1] to [0, 1] if needed
            if img.min() < 0:
                img = (img + 1) / 2
            plt.imshow(img)

        plt.title(f'Digit {i}')
        plt.axis('off')

plt.tight_layout()
plt.show()

# STUDENT ACTIVITY: Try generating the same digit with different noise seeds
# This shows the variety of styles the model can produce
print("\nSTUDENT ACTIVITY: Generating numbers with different noise seeds")

# Helper function to generate with seed
def generate_with_seed(number, seed_value=42, n_samples=10):
    torch.manual_seed(seed_value)
    return generate_number(model, number, n_samples)

# Pick a image and show many variations
# Hint select a image e.g. dog  # Change this to any other in the dataset of subset you chose
# Hint 2 use variations = generate_with_seed
# Hint 3 use plt.figure and plt.imshow to display the variations

# Enter your code here:
digit_to_show = 7  # Change this to any digit 0-9 in your dataset

variations = generate_with_seed(digit_to_show, seed_value=42, n_samples=10)

plt.figure(figsize=(15, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    if IMG_CH == 1:
        plt.imshow(variations[i][0].cpu(), cmap='gray')
    else:
        img = variations[i].permute(1, 2, 0).cpu()
        if img.min() < 0:
            img = (img + 1) / 2
        plt.imshow(img)
    plt.title(f'Variation {i+1}')
    plt.axis('off')

plt.tight_layout()
plt.show()
```

```
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
```

```
DEBUG: UpBlock - INPUT x shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
Streaming output truncated to the last 5000 lines.
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before upsample x shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - After upsample x shape: torch.Size([4, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x shape: torch.Size([4, 64, 28, 28]), skip_shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_H=28
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([4, 64, 28, 28]), skip_shape=torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([4, 128, 7, 7]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([4, 128, 7, 7])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([4, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x shape: torch.Size([4, 128, 14, 14]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_H=14
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([4, 128, 14, 14]), skip_shape=torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([4, 64, 14, 14]), skip_shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - After upsample x_shape: torch.Size([4, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([4, 64, 28, 28]), skip_shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_H=28
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([4, 64, 28, 28]), skip_shape=torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - INPUT x shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([4, 128, 7, 7]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([4, 128, 7, 7])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([4, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([4, 128, 14, 14]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_H=14
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([4, 128, 14, 14]), skip_shape=torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - INPUT x
```

```
        images = []
        images.append(x[0].cpu())

        # Remove noise step by step
        for t in range(n_steps-1, -1, -1):
            t_batch = torch.full((1,), t).to(device)
            x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

            if t in steps_to_show:
                images.append(x[0].cpu())

        # Show the progression
        plt.figure(figsize=(20, 3))
        for i, img in enumerate(images):
            plt.subplot(1, len(images), i+1)
            if IMG_CH == 1:
                plt.imshow(img[0], cmap='gray')
            else:
                img = img.permute(1, 2, 0)
                if img.min() < 0:
                    img = (img + 1) / 2
                plt.imshow(img)
            step = n_steps if i == 0 else steps_to_show[i-1]
            plt.title(f'Step {step}')
            plt.axis('off')
        plt.show()

# Show generation process for a few numbers
for number in [0, 3, 7]:
    print(f"\nGenerating number {number}:")
    visualize_generation_steps(model, number)
```

```
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([4, 128, 14, 14]), skip_shape=torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 64, 14, 14]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([4, 64, 14, 14]), skip_shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - After upsample x_shape: torch.Size([4, 64, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([4, 64, 28, 28]), skip_shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_H=28
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x_shape=torch.Size([4, 64, 28, 28]), skip_shape=torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - INPUT x_shape: torch.Size([1, 128, 7, 7]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x_shape: torch.Size([4, 128, 7, 7]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - Before upsample x_shape: torch.Size([4, 128, 7, 7])
DEBUG: UpBlock - After upsample x_shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - After upsample x_shape: torch.Size([4, 128, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before concat, after upsample/x_shape: torch.Size([4, 128, 14, 14]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_H=14
DEBUG: UpBlock - target_h=14, current_h=14
```

```
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 64, 28, 28])
Streaming output truncated to the last 5000 lines.
DEBUG: UpBlock - Before concat - after upsample/x shape: torch.Size([4, 64, 28, 28]), skip shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - target_t=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 64, 28, 28]), skip shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([4, 64, 28, 28]), skip shape=torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - Before upsample/x shape=torch.Size([1, 128, 7, 7]), skip shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 128, 7, 7]), skip shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - After upsample/x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([4, 128, 14, 14]), skip shape=torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - Before upsample/x shape=torch.Size([1, 128, 14, 14]), skip shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 64, 14, 14]), skip shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - After upsample/x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After upsample x shape: torch.Size([4, 64, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([4, 128, 14, 14]), skip shape=torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - Before upsample/x shape=torch.Size([1, 128, 7, 7]), skip shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 128, 7, 7]), skip shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - After upsample/x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - Before upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - After upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([4, 128, 14, 14]), skip shape=torch.Size([4, 32, 28, 28
```

```python
    # Install dependencies first - these help CLIP process text and images
    print("Installing CLIP dependencies...")
    !pip install -q ftfy regex tqdm

    # Install CLIP from GitHub
    print("Installing CLIP from GitHub repository...")
    !pip install -q git+https://github.com/openai/CLIP.git

    # Import and verify CLIP is working
    print("Importing CLIP...")
    import clip

    # Test that CLIP is functioning
    models = clip.available_models()
    print(f"✓ CLIP installation successful! Available models: {models}")
    clip_available = True

except ImportError:
    print("❌ Error importing CLIP. Installation might have failed.")
    print("Try manually running: !pip install git+https://github.com/openai/CLIP.git")
    print("If you're in a Colab notebook, try restarting the runtime after installation.")

except Exception as e:
    print(f"❌ Error during CLIP setup: {e}")
    print("Some CLIP functionality may not work correctly.")

# Provide guidance based on installation result
if clip_available:
    print("\nCLIP is now available for evaluating your generated images!")
else:
    print("\nWARNING: CLIP installation failed. We'll skip the CLIP evaluation parts.")

# Import necessary libraries
import functools
import torch.nn.functional as F
```

```
DEBUG: UpBlock = IMMEDIATELY before concat: x shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([4, 32, 28, 28])
DEBUG: UpBlock = Before concat, after upsample, x shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1
```

```
# CLIP (Contrastive Language-Image Pre-training) is a neural network that connects
# vision and language. It was trained on 400 million image-text pairs to understand
# the relationship between images and their descriptions.
# We use it here as an "evaluation judge" to assess our generated images.

# Load CLIP model with error handling
try:
    # Load the ViT-B/32 CLIP model (Vision Transformer-based)
    clip_model, clip_preprocess = clip.load("ViT-B/32", device=device)
    print(f"✓ Successfully loaded CLIP model: {clip_model.visual.__class__.__name__}")
except Exception as e:
    print(f"❌ Failed to load CLIP model: {e}")
    clip_available = False
    # Instead of raising an error, we'll continue with degraded functionality
    print("CLIP evaluation will be skipped. Generated images will still be displayed but without quality scores.")

def evaluate_with_clip(images, target_number, max_batch_size=16):
    """
    Use CLIP to evaluate generated images by measuring how well they match textual descriptions.

    This function acts like an "automatic critic" for our generated digits by measuring:
    1. How well they match the description of a handwritten digit
    2. How clear and well-formed they appear to be
    3. Whether they appear blurry or poorly formed

    The evaluation process works by:
    - Converting our images to a format CLIP understands
    - Creating text prompts that describe the qualities we want to measure
    - Computing similarity scores between images and these text descriptions
    - Returning normalized scores (probabilities) for each quality

    Args:
        images (torch.Tensor): Batch of generated images [batch_size, channels, height, width]
        target_number (int): The specific digit (0-9) the images should represent
        max_batch_size (int): Maximum images to process at once (prevents GPU out-of-memory errors)

    Returns:
        torch.Tensor: Similarity scores tensor of shape [batch_size, 3] with scores for:
                      [good handwritten digit, clear digit, blurry digit]
                      Each row sums to 1.0 (as probabilities)
    """
    # If CLIP isn't available, return placeholder scores
    if not clip_available:
        print("⚠️ CLIP not available. Returning default scores.")
        # Equal probabilities (0.33 for each category)
        return torch.ones(len(images), 3).to(device) / 3

    try:
        # For large batches, we process in chunks to avoid memory issues
        # This is crucial when working with big images or many samples
        if len(images) > max_batch_size:
            all_similarities = []

            # Process images in manageable chunks
            for i in range(0, len(images), max_batch_size):
                print(f"Processing CLIP batch {i//max_batch_size + 1}/{(len(images)-1)//max_batch_size + 1}")
                batch = images[i:i+max_batch_size]

                # Use context managers for efficiency and memory management:
                # - torch.no_grad(): disables gradient tracking (not needed for evaluation)
                # - torch.cuda.amp.autocast(): uses mixed precision to reduce memory usage
                with torch.no_grad(), torch.cuda.amp.autocast():
                    batch_similarities = _process_clip_batch(batch, target_number)
                    all_similarities.append(batch_similarities)

                # Explicitly free GPU memory between batches
                # This helps prevent cumulative memory buildup that could cause crashes
                torch.cuda.empty_cache()

            # Combine results from all batches into a single tensor
            return torch.cat(all_similarities, dim=0)
        else:
            # For small batches, process all at once
            with torch.no_grad(), torch.cuda.amp.autocast():
                return _process_clip_batch(images, target_number)

    except Exception as e:
```

```
        # If anything goes wrong, log the error but don't crash
        print(f"❌ Error in CLIP evaluation: {e}")
        print(f"Traceback: {traceback.format_exc()}")
        # Return default scores so the rest of the notebook can continue
        return torch.ones(len(images), 3).to(device) / 3

def _process_clip_batch(images, target_number):
    """
    Core CLIP processing function that computes similarity between images and text descriptions.

    This function handles the technical details of:
    1. Preparing relevant text prompts for evaluation
    2. Preprocessing images to CLIP's required format
    3. Extracting feature embeddings from both images and text
    4. Computing similarity scores between these embeddings

    The function includes advanced error handling for GPU memory issues,
    automatically reducing batch size if out-of-memory errors occur.

    Args:
        images (torch.Tensor): Batch of images to evaluate
        target_number (int): The digit these images should represent

    Returns:
        torch.Tensor: Normalized similarity scores between images and text descriptions
    """
    try:
        # Create text descriptions (prompts) to evaluate our generated digits
        # We check three distinct qualities:
        # 1. If it looks like a handwritten example of the target digit
        # 2. If it appears clear and well-formed
        # 3. If it appears blurry or poorly formed (negative case)
        text_inputs = torch.cat([
            clip.tokenize(f"A handwritten number {target_number}"),
            clip.tokenize(f"A clear, well-written digit {target_number}"),
            clip.tokenize(f"A blurry or unclear number")
        ]).to(device)

        # Process images for CLIP, which requires specific formatting:

        # 1. Handle different channel configurations (dataset-dependent)
        if IMG_CH == 1:
            # CLIP expects RGB images, so we repeat the grayscale channel 3 times
            # For example, MNIST/Fashion-MNIST are grayscale (1-channel)
            images_rgb = images.repeat(1, 3, 1, 1)
        else:
            # For RGB datasets like CIFAR-10/CelebA, we can use as-is
            images_rgb = images

        # 2. Normalize pixel values to [0,1] range if needed
        # Different datasets may have different normalization ranges
        if images_rgb.min() < 0:  # If normalized to [-1,1] range
            images_rgb = (images_rgb + 1) / 2  # Convert to [0,1] range

        # 3. Resize images to CLIP's expected input size (224x224 pixels)
        # CLIP was trained on this specific resolution
        resized_images = F.interpolate(images_rgb, size=(224, 224),
                                       mode='bilinear', align_corners=False)

        # Extract feature embeddings from both images and text prompts
        # These are high-dimensional vectors representing the content
        image_features = clip_model.encode_image(resized_images)
        text_features = clip_model.encode_text(text_inputs)

        # Normalize feature vectors to unit length (for cosine similarity)
        # This ensures we're measuring direction, not magnitude
        image_features = image_features / image_features.norm(dim=-1, keepdim=True)
        text_features = text_features / text_features.norm(dim=-1, keepdim=True)

        # Calculate similarity scores between image and text features
        # The matrix multiplication computes all pairwise dot products at once
        # Multiplying by 100 scales to percentage-like values before applying softmax
        similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)

        return similarity

    except RuntimeError as e:
```

```
        # Special handling for CUDA out-of-memory errors
        if "out of memory" in str(e):
            # Free GPU memory immediately
            torch.cuda.empty_cache()

            # If we're already at batch size 1, we can't reduce further
            if len(images) <= 1:
                print("❌ Out of memory even with batch size 1. Cannot process.")
                return torch.ones(len(images), 3).to(device) / 3

            # Adaptive batch size reduction - recursively try with smaller batches
            # This is an advanced technique to handle limited GPU memory gracefully
            half_size = len(images) // 2
            print(f"⚠️ Out of memory. Reducing batch size to {half_size}.")

            # Process each half separately and combine results
            # This recursive approach will keep splitting until processing succeeds
            first_half = _process_clip_batch(images[:half_size], target_number)
            second_half = _process_clip_batch(images[half_size:], target_number)

            # Combine results from both halves
            return torch.cat([first_half, second_half], dim=0)

        # For other errors, propagate upward
        raise e

#==============================================================================
# CLIP Evaluation - Generate and Analyze Sample Digits
#==============================================================================
# This section demonstrates how to use CLIP to evaluate generated digits
# We'll generate examples of all ten digits and visualize the quality scores

try:
    for number in range(10):
        print(f"\nGenerating and evaluating number {number}...")

        # Generate 4 different variations of the current digit
        samples = generate_number(model, number, n_samples=4)

        # Evaluate quality with CLIP (without tracking gradients for efficiency)
        with torch.no_grad():
            similarities = evaluate_with_clip(samples, number)

        # Create a figure to display results
        plt.figure(figsize=(15, 3))

        # Show each sample with its CLIP quality scores
        for i in range(4):
            plt.subplot(1, 4, i+1)

            # Display the image with appropriate formatting based on dataset type
            if IMG_CH == 1:  # Grayscale images (MNIST, Fashion-MNIST)
                plt.imshow(samples[i][0].cpu(), cmap='gray')
            else:  # Color images (CIFAR-10, CelebA)
                img = samples[i].permute(1, 2, 0).cpu()  # Change format for matplotlib
                if img.min() < 0:  # Handle [-1,1] normalization
                    img = (img + 1) / 2  # Convert to [0,1] range
                plt.imshow(img)

            # Extract individual quality scores for display
            # These represent how confidently CLIP associates the image with each description
            good_score = similarities[i][0].item() * 100  # Handwritten quality
            clear_score = similarities[i][1].item() * 100  # Clarity quality
            blur_score = similarities[i][2].item() * 100   # Blurriness assessment

            # Color-code the title based on highest score category:
            # - Green: if either "good handwritten" or "clear" score is highest
            # - Red: if "blurry" score is highest (poor quality)
            max_score_idx = torch.argmax(similarities[i]).item()
            title_color = 'green' if max_score_idx < 2 else 'red'

            # Show scores in the plot title
            plt.title(f'Number {number}\nGood: {good_score:.0f}%\nClear: {clear_score:.0f}%\nBlurry: {blur_score:.0f}%',
                      color=title_color)
            plt.axis('off')

        plt.tight_layout()
```

```
        plt.show()
        plt.close()  # Properly close figure to prevent memory leaks

        # Clean up GPU memory after processing each number
        # This is especially important for resource-constrained environments
        torch.cuda.empty_cache()

except Exception as e:
    # Comprehensive error handling to help students debug issues
    print(f"❌ Error in generation and evaluation loop: {e}")
    print("Detailed error information:")
    import traceback
    traceback.print_exc()

    # Clean up resources even if we encounter an error
    if torch.cuda.is_available():
        print("Clearing GPU cache...")
        torch.cuda.empty_cache()

#==============================================================================
# STUDENT ACTIVITY: Exploring CLIP Evaluation
#==============================================================================
# This section provides code templates for students to experiment with
# evaluating larger batches of generated digits using CLIP.

print("\nSTUDENT ACTIVITY:")
print("Try the code below to evaluate a larger sample of a specific digit")
print("""
# Example: Generate and evaluate 10 examples of the digit 6
# digit = 6
# samples = generate_number(model, digit, n_samples=10)
# similarities = evaluate_with_clip(samples, digit)
#
# # Calculate what percentage of samples CLIP considers "good quality"
# # (either "good handwritten" or "clear" score exceeds "blurry" score)
# good_or_clear = (similarities[:,0] + similarities[:,1] > similarities[:,2]).float().mean()
# print(f"CLIP recognized {good_or_clear.item()*100:.1f}% of the digits as good examples of {digit}")
#
# # Display a grid of samples with their quality scores
# plt.figure(figsize=(15, 8))
# for i in range(len(samples)):
#     plt.subplot(2, 5, i+1)
#     plt.imshow(samples[i][0].cpu(), cmap='gray')
#     quality = "Good" if similarities[i,0] + similarities[i,1] > similarities[i,2] else "Poor"
#     plt.title(f"Sample {i+1}: {quality}", color='green' if quality == "Good" else 'red')
#     plt.axis('off')
# plt.tight_layout()
# plt.show()
""")
```

```
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 64, 14, 14]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - target_h=14, current_h=14
DEBUG: UpBlock - Before×upsample x shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - After×upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 64, 14, 14]), skip_shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - Before concat: after upsample/x shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x shape: torch.Size([4, 128, 14, 14])
DEBUG: UpBlock - target_h=28, current_h=14
DEBUG: UpBlock - Before×upsample x shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - After upsample x shape: torch.Size([4, 64, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip_shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - After×upsample x shape: torch.Size([1, 128, 14, 14])
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 128, 7, 7]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - Before concat: after upsample/x shape: torch.Size([1, 64, 28, 28]), skip_shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x shape: torch.Size([4, 128, 7, 7])
DEBUG: UpBlock - target_h=14, current_h=7
DEBUG: UpBlock - Before×upsample x shape: torch.Size([1, 128, 7, 7])
DEBUG: UpBlock - After upsample x shape: torch.Size([4, 128, 14, 14])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 64, 28, 28]), skip_shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - After×upsample x shape: torch.Size([1, 64, 28, 28])
DEBUG: UpBlock - INPUT x shape: torch.Size([4, 128, 7, 7]), skip_shape: torch.Size([4, 64, 14, 14])
DEBUG: UpBlock - Before concat: after upsample/x shape: torch.Size([1, 128, 14, 14]), skip_shape: torch.Size([4, 64, 14, 14
```

```
DEBUG: UpBlock - BeforeConcat: x shape=torch.Size([1, 128, 7, 7]), skip shape: torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before upsample x shape: torch.Size([4, 648, 14, 14])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - BeforeConcat: x shape=torch.Size([1, 64, 28, 28]), skip shape: torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - Before upsample x shape: torch.Size([4, 648, 14, 14])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - After upsample x shape: torch.Size([4, 648, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 128, 14, 14]), skip shape=torch.Size([1, 64, 14, 14])
DEBUG: UpBlock - Before concat, after upsample x shape: torch.Size([4, 648, 28, 28]), skip shape: torch.Size([4, 32, 28, 28])
DEBUG: UpBlock - IMMEDIATELY before concat: x shape=torch.Size([1, 64, 28, 28]), skip shape=torch.Size([1, 32, 28, 28])
DEBUG: UpBlock - target_h=28, current_h=28
DEBUG: UpBlock - BeforeConcat: x shape=torch.Size([1, 128, 7, 7]), skip shape: torch.Size([1, 64, 14, 14
```

```
    # Define a single "quality score":
    # higher when "good" + "clear" is large and "blurry" is small
    quality_scores = similarities[:, 0] + similarities[:, 1] - similarities[:, 2]

    # 3. Get indices of top-3 samples
    topk = torch.topk(quality_scores, k=3)
    top_scores = topk.values
    top_indices = topk.indices
    all_top_indices[digit] = top_indices.cpu().tolist()

    print(" Top-3 CLIP quality scores:", [f"{s.item():.3f}" for s in top_scores])

    # 4. Plot all 10 with top-3 highlighted
    plt.figure(figsize=(15, 6))
    for i in range(n_samples):
        plt.subplot(2, 5, i + 1)

        if IMG_CH == 1:
            img = samples[i][0].cpu()
            plt.imshow(img, cmap='gray')
        else:
            img = samples[i].permute(1, 2, 0).cpu()
            if img.min() < 0:
                img = (img + 1) / 2
            plt.imshow(img)

        is_top = i in top_indices
        border_color = 'green' if is_top else 'black'
        plt.title(f"#{i}  Q={quality_scores[i].item():.2f}",
                  color='green' if is_top else 'black',
                  fontsize=8)
        plt.axis('off')
        # draw a colored frame
        for spine in plt.gca().spines.values():
            spine.set_edgecolor(border_color)
            spine.set_linewidth(2 if is_top else 0.5)

    plt.suptitle(f"Digit {digit} – CLIP Top-3 Highlighted", fontsize=14)
    plt.tight_layout()
    plt.show()

print("\nSummary: indices of CLIP-selected top-3 samples per digit:")
for d, idxs in all_top_indices.items():
    print(f" Digit {d}: top indices {idxs}")
```

Streaming output truncated to the last 5000 lines.